0

Перебор

циклы, itertools и дерево вариантов

Занятие 39

0

Вспомним биты

Три лампочки. Каждая либо горит, либо нет. Сколько состояний?

```
for n in range(8):
    print(format(n, "03b"))
```

```
000 001 010 011 100 101 110 111
```

У каждой два состояния: 2 × 2 × 2 = 8

А если цифр три?

Код из трёх цифр. На каждом месте: `0`, `1` или `2`.

```
for a in "012":
    for b in "012":
        for c in "012":
            print(a + b + c)
```

Что выведется сначала? `000`, `001`, `002`, ...

Сколько вариантов?

первое место	второе	третье
0, 1, 2	0, 1, 2	0, 1, 2

$3 \times 3 \times 3 = 27$

- каждый код получается ровно из одного набора `a, b, c`
- символы алфавита разные - один и тот же код дважды не появится

Оставим нужные

Теперь нужны только коды с суммой цифр 3.

```
for a in "012":
    for b in "012":
        for c in "012":
            if int(a) + int(b) + int(c) == 3:
                print(a + b + c)
```

```
012 021 102 111 120 201 210
```

Циклы строят варианты. `if` решает, какие оставить.

15

А если длина меняется?

Три цифры - три цикла. Пять цифр - пять циклов?

```
def make_codes(alphabet: str, length: int):
    # length узнаем только при вызове
    pass
```

Хочется сказать: повтори выбор length раз

`itertools`

- модуль стандартной библиотеки Python - устанавливать ничего не нужно
- готовые инструменты для перебора: коды, перестановки, сочетания
- выдают результаты по одному - их удобно обходить циклом `for`

Начнём с `product`: он повторяет выбор для каждой позиции.

product

```
from itertools import product

for chars in product("01", repeat=3):
    word = "".join(chars)
    print(word)
```

`("0", "0", "1")` → `"001"`

- `repeat=3` - три места, на каждом выбираем из `"01"`
- вместо 3 можно написать `length`

Считать или хранить?

```
from itertools import product

count = 0
for chars in product("012", repeat=5):
    if chars[0] != "0":
        count += 1
print(count)  # 162
```

- `product` выдаёт варианты по одному
- если нужен только ответ «сколько», список всех кодов не нужен

Две единицы, не рядом

Код из четырёх битов: ровно две единицы, но `11` быть не должно.

```
from itertools import product

for chars in product("01", repeat=4):
    word = "".join(chars)
    if word.count("1") == 2 and "11" not in word:
        print(word)
```

```
0101 1001 1010
```

Нужны оба условия - соединяем через `and`.

Теперь сам

- буквы `A, B, C`, длина слова 3
- буква `A` встречается ровно один раз
- одинаковые буквы не стоят рядом

Проверка на длине 2: `AB AC BA CA`. На длине 3 - 8 слов.

`practice/39-bruteforce-itertools/7_words.py`

30

Код, места или команда?

выбираем двоих из A, B, C	что считается разным?	всего
код	AA можно; AB и BA разные	9
призовые места	AA нельзя; AB и BA разные	6
команда	AA нельзя; AB и BA - одно	3

Два вопроса: важен порядок? можно брать повторно?

permutations

```
from itertools import permutations

for pair in permutations("ABC", 2):
    print("".join(pair))
```

```
AB AC BA BC CA CB
```

- `AB` и `BA` - разные расстановки
- одну позицию входа дважды не берём; одинаковые буквы во входе могут дать дубли

combinations

```
from itertools import combinations

for team in combinations("ABC", 2):
    print("".join(team))
```

```
AB AC BC
```

- `AB` и `BA` - одна команда, второй раз не выдаём
- буквы идут в порядке входа; сам вход не сортируется

Что выбрать?

задача	порядок	повторения	инструмент
PIN-код	важен	можно	`product`
призовые места	важен	нельзя	`permutations`
команда	не важен	нельзя	`combinations`

Сначала правила задачи, потом название функции.

Пары с суммой 6

Есть числа `[1, 2, 3, 4, 5]`. Выбери пары с суммой `6`.

- `(1, 5)` и `(2, 4)` подходят
- `(5, 1)` второй раз не нужен
- `(3, 3)` нельзя - тройка в списке одна

`practice/39-bruteforce-itertools/10_pair_sum.py`

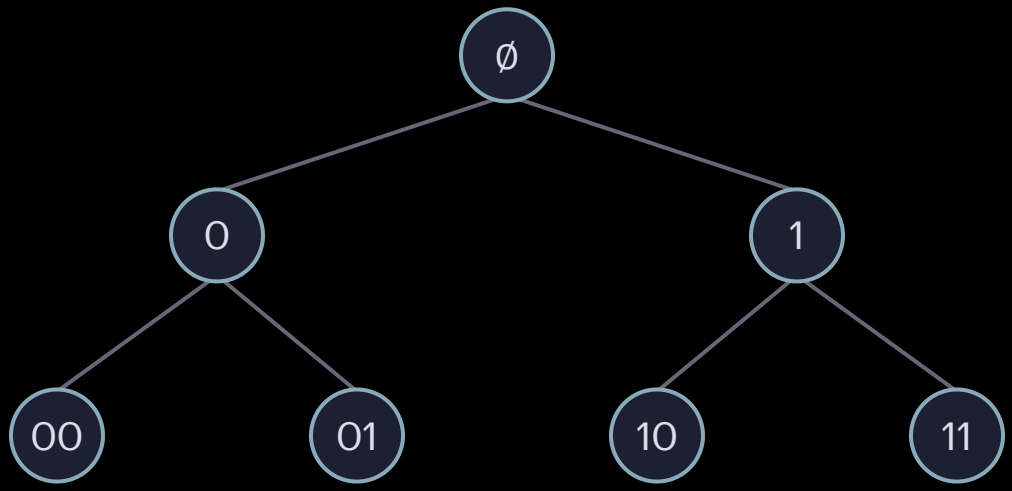
45

А сколько работы?

длина двоичного кода	вариантов
10	1 024
20	1 048 576
30	1 073 741 824

Код короткий, а вариантов всё равно 2^n.

Это же дерево



- вершина - уже выбранное начало строки, префикс
- ребро - дописать один символ; лист на глубине n - готовый код

Такое дерево мы умеем обходить: DFS.

Сюда можно не идти

Запрещены соседние единицы. А начало строки уже `011`.

что допишем	что получится
0…	`0110...`
1…	`0111...`

Ни одно продолжение не уберёт `11`. Можно отсечь всю ветку.

Следующий урок: строим строку по частям с помощью рекурсии.

Домашнее задание

Устно: код длины 5 из ABC, ровно две A. Что перебираем? Что проверяем?

- двоичные коды длины `n`: посчитать те, где ровно `k` единиц
- то же самое, но единицы не стоят рядом
- бонус: команды из `k` разных чисел с заданной суммой

Что запомнить

- сначала перебираем варианты, потом отбираем через `if`
- `product`, `permutations`, `combinations` - разные правила выбора
- число вариантов нужно оценить до запуска
- перебор можно представить деревом и не заходить в плохие ветки